

Claudio - Gramatica ampliada con reglas semanticas

Quiz 4 / Entrega 4 - Compiladores. Lenguaje fuente: Claudio (espanol) -> Swift.

Convencion visual: las producciones sintacticas originales se presentan en rojo. Las reglas semanticas agregadas se presentan en texto negro sobre fondo ambar.

Resumen de reglas implementadas

ID	Regla	Nodo gramatical	Atributos principales	Dominio / restriccion
SEM-1	Declaracion duplicada	decl_variable, def_funcion, parametros, clases	id.nombre sintetizado; ambito heredado	nombre not in tablaSimbolos[ambitoActual]
SEM-2	Identificador no declarado	sent_id, expresiones	id.tipo sintetizado desde tabla	nombre in tablaSimbolos
SEM-3	Constante reasignada	sent_id	id.inmutable heredado desde tabla	immutable == false para asignaciones
SEM-4	Tipo incompatible en declaracion	decl_variable	decl.tipo sintetizado; expr.tipo sintetizado	Tipos iguales o entero -> real
SEM-5	Tipo incompatible en asignacion	sent_id	id.tipo heredado; expr.tipo sintetizado	Tipos compatibles con el tipo declarado
SEM-6	Condicion no booleana	sent_si, sent_mientras	cond.tipo sintetizado	cond.tipo == booleano
SEM-7	Limites no numericos en para	sent_para	desde.tipo, hasta.tipo, paso.tipo sintetizados	Cada tipo debe pertenecer a {entero, real}

Producciones ampliadas

Declaracion de variable

<decl_variable> -> (var | sea) <tipo> ID = <expresion>

SEM-1 - Declaracion duplicada en el mismo ambito

- **Atributos:** ID.nombre sintetizado; ambitoActual heredado desde la pila de ambitos.
- **Dominio:** ID.nombre no debe existir en tablaSimbolos[ambitoActual].
- **Accion:** consultar tabla de simbolos; si ya existe, reportar fila, columna, lexema y ubicacion de la primera declaracion; continuar analisis.

SEM-4 - Tipo compatible en inicializacion

- **Atributos:** decl.tipo sintetizado desde <tipo>; expr.tipo sintetizado por inferencia.
- **Dominio:** decl.tipo == expr.tipo o decl.tipo == real y expr.tipo == entero.
- **Accion:** inferir tipo de la expresion; comparar contra el tipo declarado; si falla, reportar error y registrar el simbolo para permitir errores posteriores.

Sentencia por identificador

<sent_id> -> ID = <expresion> | ID (<argumentos>) | ID . ID <sufijo_id>

SEM-2 - Uso de identificador previamente declarado

- **Atributos:** ID.nombre sintetizado; ID.tipo sintetizado desde tabla.
- **Dominio:** ID.nombre debe existir en algun ambito activo de la tabla de simbolos.
- **Accion:** buscar desde el ambito mas interno al global; si no existe, reportar fila, columna, lexema y sugerencia de declaracion.

SEM-3 - Las constantes declaradas con sea no se reasignan

- **Atributos:** ID.inmutable heredado desde la entrada de tabla; ID.nombre sintetizado.
- **Dominio:** si la produccion es asignacion, ID.inmutable debe ser false.
- **Accion:** leer atributo de tabla; si es constante, reportar error y mantener el analisis de la expresion asignada.

SEM-5 - Tipo compatible en asignacion

- **Atributos:** ID.tipo heredado desde tabla; expr.tipo sintetizado.
- **Dominio:** mismo criterio de compatibilidad de SEM-4.
- **Accion:** inferir tipo de <expresion>; comparar; reportar el tipo esperado y el tipo encontrado.

Condicional y ciclo mientras

<sent_si> -> si <expresion> entonces <bloque> <rama_sino> fin_si

<sent_mientras> -> mientras <expresion> hacer <bloque> fin_mientras

SEM-6 - La condicion debe ser booleana

- **Atributos:** cond.tipo sintetizado por inferencia.
- **Dominio:** cond.tipo in {booleano} cuando el tipo puede determinarse.
- **Accion:** si la condicion se infiere como entero, real o cadena, reportar error en el token si o mientras.

Ciclo para

<sent_para> -> para ID desde <expresion> hasta <expresion> <paso_opt> hacer <bloque> fin_para

<paso_opt> -> paso <expresion> | epsilon

SEM-7 - Limites y paso numericos

- **Atributos:** desde.tipo, hasta.tipo y paso.tipo sintetizados.
- **Dominio:** cada atributo debe pertenecer a {entero, real}.
- **Accion:** declarar la variable del ciclo como entero en un ambito local; validar limites y paso; reportar cada violacion sin detener el recorrido.

Tabla de simbolos

La implementacion mantiene una pila de ambitos. Cada entrada conserva: nombre, tipo, inmutable, inicializado, ambito, fila y columna. La busqueda se realiza del ambito mas interno hacia el global.

Bonus IA con OpenAI

Se implementa la modalidad A: los errores semanticos deterministas se envian a OpenAI como capa adicional de explicacion. La respuesta agrega explicacion, correccion sugerida, mini ejemplo y confianza. Si OPENAI_API_KEY no esta configurada, el compilador conserva la sugerencia deterministica y responde estado=no_disponible.

Ejemplo de validacion

```
var entero x = "hola"
var entero x = 2
x = "texto"
sea entero c = 1
c = 2
```

```
imprimir(fantasma)
si x entonces
    imprimir(x)
fin_si
para i desde "a" hasta 10 paso falso hacer
    imprimir(i)
fin_para
```

Este ejemplo dispara SEM-1, SEM-2, SEM-3, SEM-4, SEM-5, SEM-6 y SEM-7 en una sola ejecucion, demostrando continuacion del analisis despues de cada error.